

Progettazione di Applicazioni Mobili

Flutter Programming

Widgets – Style, Views, etc.

Stefano Burigat

Dipartimento di Scienze Matematiche, Informatiche e
Fisiche - Università di Udine

stefano.burigat@uniud.it



Styling widgets: Theme and ThemeData

- The MaterialApp widget has a theme property that can be used to declare the theme to apply for the entire application using the ThemeData widget
- The Theme widget comes into play when you want to override that application-level theme for just some subset of widgets
- There are two choices when dealing with the Theme widget: extending the parent theme or building a new one

Styling widgets: Theme and ThemeData

- Extending the parent theme (whatever theme is nearest when traversing up the widget tree) is good when you only want to change a subset of elements

```
Theme(  
  data: Theme.of(context).copyWith(splashColor: Colors.yellow),  
  child: const FloatingActionButton(  
    onPressed: null,  
    child: Icon(Icons.add),  
  ),  
);
```

- The Theme.of() method is basically how you find the nearest theme to this widget
 - even if you don't explicitly define a theme anywhere there's a default theme automatically that will be found

Styling widgets: Theme and ThemeData

- The Theme.of() method returns a ThemeData object, which has a copyWith() method
 - The copyWith() method returns a new ThemeData object but with whatever properties you pass to it overriding what was in it before
- To create a whole new Theme is even easier: you just create a new instance of ThemeData and define the properties you want

```
Theme(  
  // Create a unique theme with `ThemeData`  
  data: ThemeData(  
    splashColor: Colors.yellow,  
  ),  
  child: FloatingActionButton(  
    onPressed: () {},  
    child: const Icon(Icons.add),  
  ),  
);
```

Styling widgets: Theme and ThemeData

- Once you have defined a Theme, you still must make use of it in individual widgets using the Theme.of(context) method

```
Container(  
  color: Theme.of(context).colorScheme.secondary,  
  child: Text(  
    'Text with a background color',  
    style: Theme.of(context).textTheme.headline6,  
  ),  
)
```

- The Theme.of(context) method looks up the widget tree and returns the nearest Theme in the tree. If you have a standalone Theme defined above your widget, that's returned. If not, the app's theme is returned.

Styling widgets: Opacity

- The Opacity widget makes its child transparent by a specified amount
- The following example makes the text semitransparent (25% opaque)

```
Opacity(opacity: .25, child : Text("Faded") )
```

Styling widgets: DecoratedBox

- The DecoratedBox widget paints a decoration onto its child box-like container
- Going along with DecoratedBox, nearly always, is the BoxDecoration widget, which defines the decoration you want

```
class Home extends StatelessWidget {  
  @override  
  Widget build(BuildContext inContext) {  
    return Scaffold(  
      body: Center(  
        child: DecoratedBox(  
          decoration: BoxDecoration(  
            gradient: LinearGradient(  
              begin: Alignment.topCenter,  
              end: Alignment.bottomCenter,  
              colors: [Color(0xFF000000), Color(0xFFFF0000)],  
              tileMode: TileMode.repeated)), // LinearGradient, BoxDecoration  
          child: Container(  
            width: 100,  
            height: 100,  
            child: Text("Hello", style: TextStyle(color: Colors.white))),  
        ),  
      ),  
    );  
  }  
}
```

The BoxDecoration widget provides a way to decorate with colors, or with images (to have a background image applied to the Container behind the Text, for example) or play with the borders (rounded corners, for example) and apply shadows and gradients

Styling widgets: Transform

- The Transform widget applies some sort of geometric transformation on its child element

```
class Home extends StatelessWidget {  
  @override  
  Widget build(BuildContext inContext) {  
    return Scaffold(  
      body: Center(  
        child: Container(  
          color: Colors.yellow,  
          child: Transform(  
            alignment: Alignment.bottomLeft,  
            transform: Matrix4.skewY(0.4)..rotateZ(-3 / 12.0),  
            child: Container(  
              padding: const EdgeInsets.all(12.0),  
              color: Colors.red,  
              child: Text("Eat at Joe's!")))), // Container,  
          ); // Scaffold  
        }  
      }  
    }
```

This code rotates and skews a red box with a yellow background that has some text in it while keeping the bottom left corner of the box pinned to its original location

In addition to the Transform constructor, there are also Transform.rotate(), Transform.scale(), and Transform.translate() that do not require matrix operations ⁸

Animations and Transitions:

AnimatedContainer

- The AnimatedContainer widget gradually changes its values over a defined period of time
 - you just tell it what the starting values are, then change to the new values and it will animate (or tween) between them as required

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatefulWidget{
  @override
  _MyApp createState() => _MyApp();
}

class _MyApp extends State {
  var _color = Colors.yellow;
  var _height = 100.0;
  var _width = 100.0;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              AnimatedContainer(
                duration: Duration(seconds: 1),
                color: _color,
                width: _width,
                height: _height), // AnimatedContainer
              RaisedButton(
                child: Text("Animate!"),
                onPressed: () {
                  _color = Colors.red;
                  _height = 200.0;
                  _width = 200.0;
                  setState(() {});
                }) // RaisedButton
            ])))); // Column, Center, Scaffold, MaterialApp
  }
}
```

Animations and Transitions:

AnimatedCrossFade

- The AnimatedCrossFade widget is a widget specifically designed to fade one element out while another fades in at the same location

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatefulWidget {
  @override
  _MyApp createState() => _MyApp();
}

class _MyApp extends State {
  var _showFirst = true;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              AnimatedCrossFade(
                duration: Duration(seconds: 2),
```

```
                firstChild:
                  FlutterLogo(style: FlutterLogoStyle.horizontal, size: 100.0),
                secondChild:
                  FlutterLogo(style: FlutterLogoStyle.stacked, size: 100.0),
                crossFadeState: _showFirst
                  ? CrossFadeState.showFirst
                  : CrossFadeState.showSecond,
              ), // AnimatedCrossFade
            ],
          ), // Column, Center, Scaffold, MaterialApp
    );
```

Animations and Transitions:

AnimatedDefaultTextStyle

- AnimatedDefaultTextStyle works very similarly to AnimatedContainer/AnimatedCrossFade and is a good choice for animating text

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatefulWidget {
  @override
  _MyApp createState() => _MyApp();
}

class _MyApp extends State {
  var _color = Colors.red;
  var _fontSize = 20.0;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              AnimatedDefaultTextStyle(
                duration: const Duration(seconds: 1),
                style: TextStyle(color: _color, fontSize: _fontSize),
                child: Text("I am some text")), // AnimatedDefaultText
              RaisedButton(
                child: Text("Enhance!"),
                onPressed: () {
                  _color = Colors.blue;
                  _fontSize = 40.0;
                  setState(() {});
                } // RaisedButton
            ]
          )
        )
      )
    ); // Column, Center, Scaffold, MaterialApp
  }
}
```

Drag and Drop

- Flutter supports the drag-and-drop interaction via two main widgets: Draggable and DragTarget

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        body: Center(  
          child: Column(  
            mainAxisAlignment: MainAxisAlignment.center,  
            children: [  
              DragTarget(  
                builder: (BuildContext context, List<String> accepted,  
                  List<dynamic> rejected) {  
                  return new Container(  
                    width: 200, height: 200, color: Colors.lightBlue); // Co  
                },  
                onAccept: (data) => print(data)), // DragTarget  
              Container(height: 50),  
              Draggable(  
                data: "I was dragged",  
                child: Container(width: 100, height: 100, color: Colors.red),  
                feedback:  
                  Container(width: 100, height: 100, color: Colors.yellow))  
            ])))); // Column, Center, Scaffold, MaterialApp  
  }  
}
```

The DragTarget is where the thing being dragged can be dropped. We need to specify the data type (if any) that this target will accept, a String in this case

In the Draggable widget, we specify the data property, which is any arbitrary data you want to provide to the DragTarget, the feedback property, which is the widget that the user will physically drag, and the child property which is the original widget (that never moves)

Drag and Drop

- When the widget specified by the feedback property is dropped onto a DragTarget, the onAccept handler function on the DragTarget fires and receives the data property value of the Draggable
- There are a host of other callbacks that can be triggered in various situations on both widgets, but probably the most useful is the onDragComplete handler, which is a function that fires when the Draggable is dropped on a DragTarget
 - This is usually the place where you'd hide the original child widget or do whatever else makes sense

Data Views: Table

- Flutter provides a handful of widgets specifically for showing lists of data to the user
- The Table widget displays elements in an organization of rows and columns
 - You can optionally define the border of the Table (by default, it will have none)
 - Then, you just need to supply some rows via children, which can be any widget or widget tree you like but it must at the top be an instance of TableRow, and the children of each is a cell, or column, in the row
 - Every row in a Table must have the same number of children
 - You can manually set the widths of the columns with the columnWidths property, and you can adjust the vertical alignment of content in each cell with the defaultVerticalAlignment property

Data Views: Table

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Column(children: [
          Container(height: 100),
          Table(
            border: TableBorder(
              top: BorderSide(width: 2),
              bottom: BorderSide(width: 2),
              left: BorderSide(width: 2),
              right: BorderSide(width: 2)), // TableBorder
            children: [
              TableRow(children: [
                Center(
                  child:
                    Padding(padding: EdgeInsets.all(10), child: Text("1")), // Center
                Center(
                  child:
                    Padding(padding: EdgeInsets.all(10), child: Text("2")), // Center
                Center(
                  child: Padding(padding: EdgeInsets.all(10), child: Text("3")) // Center
                ]) // TableRow
            ]) // Table
          ])); // Column, Scaffold, MaterialApp
  }
}
```

Data Views: DataTable

- Flutter provides the DataTable widget to display data in a tabular form
- A DataTable requires that you tell it what the columns in the table are and then what the data rows to display are
 - Each column is defined via a DataColumn instance while each row is defined with a DataRow instance which contains a collection of cells whose members are DataCell instances
 - You can supply the sortColumnIndex property to indicate which column the data is currently sorted by (this is just a visual indicator, your code is responsible for physically sorting the data)

Data Views: DataTable

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Column(children: [
          Container(height: 100),
          DataTable(sortColumnIndex: 1, columns: [
            DataColumn(label: Text("First Name")),
            DataColumn(label: Text("Last Name"))
          ], rows: [
            DataRow(cells: [
              DataCell(Text("Leia")),
              DataCell(Text("Organa"), showEditIcon: true)
            ]), // DataRow
            DataRow(cells: [DataCell(Text("Luke")), DataCell(Text("Skywalker"))]),
            DataRow(cells: [DataCell(Text("Han")), DataCell(Text("Solo"))])
          ]) // DataTable
        ])); // Column, Scaffold, MaterialApp
  }
}
```

Data Views: GridView

- The GridView widget displays a two-dimensional grid of widgets
- It can scroll in either direction according to the `scrollDirection` property (defaulting to `Axis.vertical`)
- The most common layout is generated by the `GridView.count()` constructor which shows a fixed number of elements (called tiles) in the cross axis
 - Other layouts include `GridView.extent()`, which produces a layout with tiles that have a maximum cross-axis extent, and `GridView.builder()` which can be used if you have an “infinite” number of tiles to display

Data Views: GridView

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: GridView.count(
          padding: EdgeInsets.all(4.0),
          crossAxisCount: 4,
          childAspectRatio: 1.0,
          mainAxisSpacing: 4.0,
          crossAxisSpacing: 4.0,
          children: [
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo()),
            GridTile(child: new FlutterLogo())
          ]
        )
      )
    ); // GridView.count, Scaffold, MaterialApp
  }
}
```

Data Views: ListView and ListTile

- The ListView widget is probably the most important of the data view widgets
- It's likely to be the one you'll use most often when you have a scrolling list of items to display
- The children of ListView can be anything you like, but frequently you will use a ListTile widget (several of them, to be more precise)
 - ListTile is a widget that is a single fixed-height row that contains some text and a leading or trailing icon
 - ListTile can show up to three lines of text, including a subtitle
 - The ListView can scroll vertically or horizontally depending on the setting of its scrollDirection property

Data Views: ListView and ListTile

- ListView supplies a couple of different constructors, with the default one shown in the next example
 - There is also a `ListView.builder()` constructor that uses a builder function to render the rows
 - `ListView.separated()` is also available, and will provide you a ListView with items separated by list separators you define
 - The `ListView.custom()` constructor allows you more flexibility in setting the child model to make the ListView look and work in pretty much any way you want
- There is also the `PageView` widget, which is a ListView that supports pagination

Data Views: ListView and ListTile

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(  
      home: Scaffold(  
        body: ListView(children: [  
          ListTile(leading: Icon(Icons.gif), title: Text("1")),  
          ListTile(leading: Icon(Icons.book), title: Text("2")),  
          ListTile(leading: Icon(Icons.call), title: Text("3")),  
          ListTile(leading: Icon(Icons.dns), title: Text("4")),  
          ListTile(leading: Icon(Icons.cake), title: Text("5")),  
          ListTile(leading: Icon(Icons.pets), title: Text("6")),  
          ListTile(leading: Icon(Icons.poll), title: Text("7")),  
          ListTile(leading: Icon(Icons.face), title: Text("8")),  
          ListTile(leading: Icon(Icons.home), title: Text("9")),  
          ListTile(leading: Icon(Icons.adb), title: Text("10")),  
          ListTile(leading: Icon(Icons.dvr), title: Text("11")),  
          ListTile(leading: Icon(Icons.hd), title: Text("12")),  
          ListTile(leading: Icon(Icons.toc), title: Text("3")),  
          ListTile(leading: Icon(Icons.tv), title: Text("14")),  
          ListTile(leading: Icon(Icons.help), title: Text("15"))  
        ])); // ListView, Scaffold, MaterialApp  
      }  
    }  
  }
```

CircularProgressIndicator

- CircularProgressIndicator it's just an animated circle that you can show to your users when some long-running activity is ongoing

```
CircularProgressIndicator()
```

- Useful properties of CircularProgressIndicator:
 - strokeWidth allows you to determine how thick the circle is
 - backgroundColor lets you set a different color behind the indicator
 - valueColor allows you to define the color of the circle itself

LinearProgressIndicator

- The LinearProgressIndicator shows progress as a colored line

```
LinearProgressIndicator(value : .25, backgroundColor : Colors.yellow)
```

- Useful properties of LinearProgressIndicator:
 - value is a number between zero and one that determines how much progress has been made and hence how much of the bar is colored
 - backgroundColor is the color of the portion of the bar corresponding to remaining progress
 - valueColor allows you to define the color of the portion that has completed

Icon

- The Icon widget provides a means to show a Material icon on the screen

```
Icon(Icons.radio)
```

- The Icons class contains a list of Material icons for your use
 - you can also add your own icons through custom fonts (this requires installing specific plugins or manually adding the font to your project through the pubspec.yaml file)

Image

- The Image widget provides a means to show an image of some sort
- This widget offers several different constructors, each for fetching an image from a different location
- Image.asset() can be used to load an image that is included in the app bundle itself
 - We can just pass the app path to the image like Image.asset("img/ron.jpg") but we must also tell Flutter how to find the image by adding a new section to the pubspec.yaml file under the flutter heading

```
assets:  
  - img/ron.jpg
```

Image

- Loading an image from the network is even easier since there are no assets to declare

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Image.network(
            "https://www.uniud.it/++theme++uniudine-theme/static/images/logo_small.png"),
        )); // Center, Scaffold, MaterialApp
  }
}
```

Chip

- A Chip is small visual element that is typically meant to display attributes of things, or small text, or to represent entities like users or quick actions the user can take

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Chip(
            avatar: CircleAvatar(
              backgroundImage: AssetImage("img/ron.jpg"),
              backgroundColor: Colors.grey.shade300,
            ),
            label: Text("Frank Zammetti")))); // Chip, Cent
  }
}
```

The avatar property is optional and usually shows either an image or the initials of the user

This property takes a value that is itself a Widget, so in theory, you could put anything you want here

FloatingActionButton

- The FloatingActionButton widget is a round button that floats above the main content and provides the user quick access to some primary function
- More times than not, you'll specify it as the value of the floatingActionButton property of the Scaffold widget

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        floatingActionButton: FloatingActionButton(
          backgroundColor: Colors.red,
          foregroundColor: Colors.yellow,
          child: Icon(Icons.add),
          onPressed: () {
            print("Ouch! Stop it!");
          }, // FloatingActionButton
        ),
        body: Center(child: Text("Click it!"))), //
      ),
    );
  }
}
```

Usually, the child of the FloatingActionButton will be an Icon

The backgroundColor property makes the button itself whatever color you like, and foregroundColor makes the icon or text on the button the specified color

The onPressed property is optional and if not specified will make the button disabled and unresponsive to touch

PopupMenuButton

- The PopupMenuButton widget implements the common “three-dot” menu paradigm to display a popup menu providing options to a user
- The widget has an onSelected property that is a callback function you provide which receives the value associated with the selected option

The PopupMenuButton widget uses the builder pattern to construct a list of PopupMenuItem widgets.

These widgets can have any child you deem appropriate, but most commonly it's just a Text widget.

You associate a value with each item, and then your onSelected function is passed that value and is responsible for what each does

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: PopupMenuButton(
            onSelected: (String result) {
              print(result);
            },
            itemBuilder: (BuildContext context) =>
              <PopupMenuEntry<String>>[
                PopupMenuItem(value: "copy", child: Text("Copy")),
                PopupMenuItem(value: "cut", child: Text("Cut")),
                PopupMenuItem(value: "paste", child: Text("Paste"))
              ])); // <PopupMenuEntry<String>>[], PopupMenuButton,
    );
  }
}
```